

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 5

Выполнила:

Савченко Анастасия

Группа К3341, Поток БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подключить и настроить RabbitMQ, реализовать межсервисное взаимодействие между микросервисами приложения бронирования столиков посредством очередей сообщений.

Ход работы

1. Установка и запуск RabbitMQ

Для запуска RabbitMQ использован Docker. Создан файл docker-compose.yml с образом rabbitmq:3-management, который предоставляет панель управления на порту 15672 и AMQP-порт 5672 для подключения сервисов. Контейнер запущен командой `docker compose up -d`. Проверен вход в панель управления через браузер (логин/пароль guest/guest).

2. Подключение сервисов к RabbitMQ

В микросервисы booking-service и restaurant-service добавлена библиотека amqpplib. В каждом сервисе реализована функция подключения к RabbitMQ, которая создаёт канал и объявляет очереди.

Используемые очереди:

review.created — событие создания нового отзыва

booking.cancelled — событие отмены бронирования

3. Реализация публикации сообщений (booking-service)

В booking-service добавлена функция публикации сообщений. При наступлении событий сервис отправляет JSON-сообщение в соответствующую очередь:

После успешного создания отзыва публикуется сообщение в очередь review.created с данными: идентификатор отзыва, идентификатор ресторана, оценка.

После успешной отмены бронирования публикуется сообщение в очередь `booking.cancelled` с данными: идентификатор бронирования, идентификатор столика, идентификатор ресторана.

Сообщения отправляются асинхронно, не блокируя основной поток обработки запроса. При недоступности RabbitMQ сервис продолжает работу, сообщения не теряются (при настроенном подтверждении).

4. Реализация потребления сообщений (restaurant-service)

В `restaurant-service` добавлены обработчики для каждой очереди. Сервис подписывается на очереди `review.created` и `booking.cancelled` и при получении сообщения выводит информацию в консоль.

При получении сообщения из очереди `review.created` выводится идентификатор ресторана и оценка.

При получении сообщения из очереди `booking.cancelled` выводится идентификатор брони и столика.

После обработки сообщения отправляется подтверждение (`ack`), удаляющее сообщение из очереди.

5. Тестирование

Проведено сквозное тестирование:

Запущены все четыре терминала: `auth-service`, `restaurant-service`, `booking-service`, `api-gateway`.

Через Postman создан отзыв к ресторану.

В консоли `restaurant-service` появилось сообщение: «Новый отзыв: ресторан 1, оценка 5», что подтверждает успешную доставку сообщения через RabbitMQ от `booking-service` к `restaurant-service`.

Панель управления RabbitMQ (<http://localhost:15672>) показывает созданные очереди и сообщения в них.

Вывод

В результате выполнения домашнего задания настроен брокер сообщений RabbitMQ через Docker. Реализовано асинхронное межсервисное взаимодействие: booking-service публикует события в очереди, restaurant-service их потребляет. Очереди сообщений позволяют сервисам обмениваться данными без прямых HTTP-запросов, что повышает отказоустойчивость и уменьшает связанность микросервисов. Система готова к дальнейшей контейнеризации в рамках ЛР3.